

08.07.2026

Integration des SDRAM-Controller in den PicoNut-Prozessor

Projektarbeit 1

Hermann Zoha

Matrikelnummer:

Fakultät:	Informatik
Studiengang:	Technische Informatik
Semester:	SoSe
Kurs:	PicoNut: Ein modularer RISC-V-Prozessor
Prüfer/in:	Prof. Dr. Gundolf Kiefer
Abgabedatum:	04.07.2026

Lizenzen

© Hermann Zoha 2026

Diese Arbeit mit dem Titel

Integration des SDRAM-Controller in den PicoNut-Prozessor

von Hermann Zoha steht unter folgender Lizenz:

Creative Commons Namensnennung-Nicht kommerziell-Share Alike 4.0 International



<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.de>

Sämtliche, in der Arbeit beschriebene und auf dem beigelegten Datenträger vorhandene, Ergebnisse dieser Arbeit in Form von Quelltexten, Software und Konzeptentwürfen stehen unter einer

GNU General Public License Version 3

<http://www.gnu.de/documents/gpl.de.html>

Die Typst-Vorlage dieser Arbeit unter folgendem Repository:

<https://github.com/fuchs-fabian/typst-template-aio-studi-and-thesis>

wurde erstellt von © Fabian Fuchs und unterliegt der

MIT License

<https://mit-license.org/>

Erklärung zur Projektarbeit

Ich, Hermann Zoha , versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema

Integration des SDRAM-Controller in den PicoNut-Prozessor

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe, wobei ich alle wörtlichen und sinngemäßen Zitate als solche gekennzeichnet habe. Lediglich Rechtschreib-, Grammatik- und Satzbaukurturen wurden mithilfe einer KI vorgenommen. Inhalt, Argumentation und wissenschaftliche Aussagen stammen ausschließlich von mir.

Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Technischen Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Ich bin mir bewusst, dass mit KI generierte Texte keine Garantie für die Qualität von Inhalten und Text bieten. Daher erkläre ich, dass ich KI-Werkzeuge lediglich als Hilfsmittel genutzt habe, die von KI generierten Inhalte kritisch überprüft habe und mein eigenständiger kognitiver sowie kreativer Einfluss in dieser Arbeit überwiegt. Ich versichere, dass ich die Inhalte meiner Arbeit vollständig verstanden habe und selbstständig vertreten kann.

Augsburg, den 08.07.2026.

Hermann Zoha

Inhaltsverzeichnis

LIZENZEN	2
ERKLÄRUNG ZUR PROJEKTARBEIT	2
ABBILDUNGSVERZEICHNIS	II
1. EINLEITUNG	1
2. SDRAM CONTROLLER	2
2.1. Anforderungen an die SDRAM-Controller	2
2.2. Überblick:PicoNut-System, Wishbone-Systembus und SDRAM	2
2.2.1. Piconut-System	2
2.2.2. Wishbone-Systembus	3
2.2.3. SDRAM	4
2.3. Entwurf des SDRAM-Controllers	5
2.3.1. Einfache SDRAM-Controller Zustandsmaschine	5
2.3.2. Erforderliche Befehle und Signal Zustände	6
2.3.3. Initialisierung	7
2.3.4. Schreibe- und Lese Zyklen	8
2.3.5. Refresh Zyklus	9
2.4. Implementierungsdetails des SDRAM-Controllers	9
2.4.1. Idel Zustand	11
2.4.2. Zwischenspeicher	11
2.4.3. Precharge Zustand	13
2.5. Test und Verifikation des SDRAM-Controllers	13
3. FAZIT	15
3.1. Zusammenfassung	15
3.2. Ausblick	15
LITERATUR- UND QUELLENVERZEICHNIS	16

Abbildungsverzeichnis

Abbildung 1	Aufbau des PicoNut-Systems	2
Abbildung 2	Diagramm des Wishbone-Systembus Schnittstelle.	3
Abbildung 3	Anschlüsse des SDRAMs.	4
Abbildung 4	Einfache Zustandsmaschine [1]	5
Abbildung 5	Zustände im Init Zyklus	7
Abbildung 6	Zustände im Lese- und Schreib Zyklen	8
Abbildung 7	Zustände im Refresh Zyklus	9

1. Einleitung

Das Ziel des PicoNut-Projekts [2] ist die Entwicklung eines minimalen RISC-V-Prozessors, der gleichzeitig flexibel erweiterbar ist.

Der Prozessor dient dem Einsatz in Lehre und Forschung. So können beispielsweise die Interaktion zwischen Hard- und Software sowie die Entwicklung und Integration von Modulen untersucht werden. Die Implementierung des PicoNut-Prozessors basiert auf SystemC [3], einer C++-Klassenbibliothek, die als Hardwarebeschreibungssprache dient.

Der SDRAM-Controller stellt eine Schnittstelle zwischen der CPU und einem externen SDRAM dar. Über diese Schnittstelle können Daten in den Arbeitsspeicher geschrieben und aus diesem gelesen werden. Dadurch erhält der PicoNut zusätzlichen Speicherplatz für Zwischenergebnisse und kann größere Datenmengen effizient verarbeiten.

Ziel dieser Arbeit ist die vollständige, synthesesfähige Implementierung dieses Moduls. Es soll den SDRAM initialisieren sowie Lese- und Schreibzugriffe ermöglichen. Darüber hinaus muss der SDRAM in regelmäßigen Zeitabständen automatisch aufgefrischt werden, um die gespeicherten Daten zuverlässig zu erhalten.

Der Aufbau dieser Arbeit gestaltet sich wie folgt: Zunächst werden in Abschnitt 2.1 die Anforderungen an den SDRAM-Controller erläutert. Anschließend gibt Abschnitt 2.2 einen Überblick über den PicoNut-Prozessor sowie die Anschlüsse des SDRAMs. Darauf aufbauend wird in Abschnitt 2.3 die Integration des SDRAM-Controllers in den PicoNut Peripherie beschrieben. Die eigentliche Hardwareimplementierung des SDRAM-Controllers wird anschließend in Abschnitt 2.4 vorgestellt. Abschließend werden in Abschnitt 2.5 die durchgeführten Tests und deren Ergebnisse erläutert. Die Arbeit schließt mit einer Zusammenfassung der erzielten Ergebnisse sowie einem Ausblick auf mögliche Erweiterungen in Abschnitt 3.

Diese schriftliche Ausarbeitung wurde mithilfe von Typst [4] erstellt. Typst ist ein Zeichensatzsystem, welches eine Alternative zu bekannten Systemen wie LaTeX oder Google Docs darstellt, wobei insbesondere auf die Kombination von hoher Funktionalität mit Nutzerfreundlichkeit geachtet wird.

2. SDRAM Controller

Sofern nicht anders angegeben, beziehen sich die folgenden Inhalte auf die PicoNut-Dokumentation. [5]

2.1. Anforderungen an die SDRAM-Controller

Wie bereits in der Einleitung erwähnt, ist das Ziel dieser Arbeit die Entwicklung einer Schnittstelle zu einem externen SDRAM, um dort Daten effizient speichern und lesen zu können. Diese Schnittstelle soll sowohl simulationsfähig als auch synthesesfähig sein.

Im Exposé dieses Projekts wurden folgende Ziele definiert:

- Simulation des SDRAM-Controllers
- Synthesefähige Hardwareimplementierung des SDRAM-Controllers

Insbesondere sollen die folgenden Zustände implementiert werden:

Initialisierung, Active, Read, Write, Precharge, Refresh

Die genaue Bedeutung dieser Zustände wird in Abschnitt 2.2 erläutert.

Der modulare Ansatz dieser Arbeit sieht vor, zunächst die grundlegenden Funktionen zur Kommunikation mit dem SDRAM zu implementieren. Dazu gehören die Ansteuerung des SDRAMs mit dem Systemtakt sowie die Übertragung von Daten zwischen dem PicoNut und dem externen Speicher. Gleichzeitig wird die Architektur so entworfen, dass zukünftige Erweiterungen, beispielsweise eine eigene Takterzeugung oder zusätzliche SDRAM-Funktionen, mit geringem Aufwand integriert werden können.

2.2. Überblick:PicoNut-System, Wishbone-Systembus und SDRAM

2.2.1. Piconut-System

Ein **PicoNut-System** besteht im Wesentlichen aus einem *PicoNut-Core*, dem *Wishbone-Systembus* sowie verschiedenen *Peripheriemodulen*, wie beispielsweise dem Audio- oder UART-Modul.

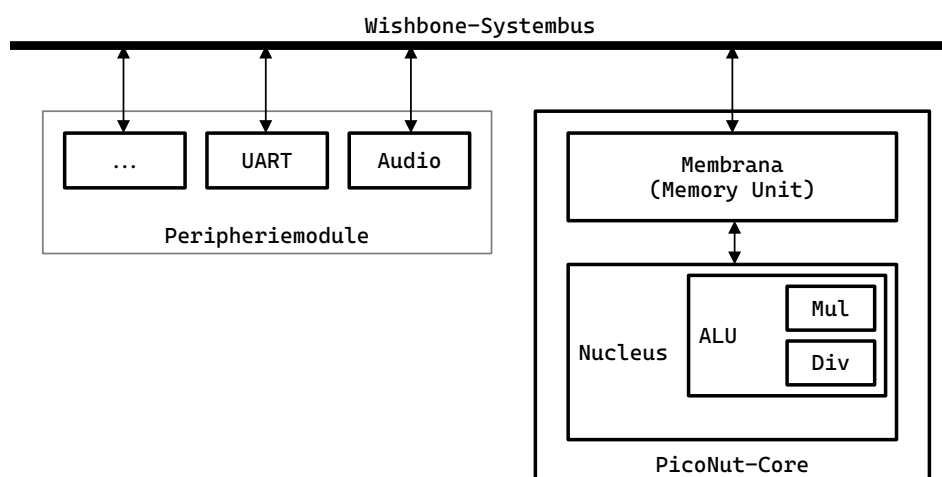


Abbildung 1: **Aufbau des PicoNut-Systems**

In Abbildung 1 wird das Zusammenspiel dieser Komponenten dargestellt. Der PicoNut-Core besteht aus den beiden Komponenten *Membrana* und *Nucleus*. Die *Membrana*

(Memory Unit) ist für den Datenaustausch zwischen dem Nucleus und dem Wishbone-Systembus verantwortlich. Im Nucleus werden die Prozessorinstruktionen ausgeführt.

Wie bereits in Abschnitt 2.1 erwähnt, soll eine Schnittstelle implementiert werden, die den Zugriff auf einen externen SDRAM ermöglicht. Hierzu müssen die von der Membrana über den Wishbone-Systembus gesendeten Signale ausgewertet werden. Anschließend werden die entsprechenden Befehle und Daten mit den erforderlichen Steuersignalen an den SDRAM weitergeleitet.

2.2.2. Wishbone-Systembus

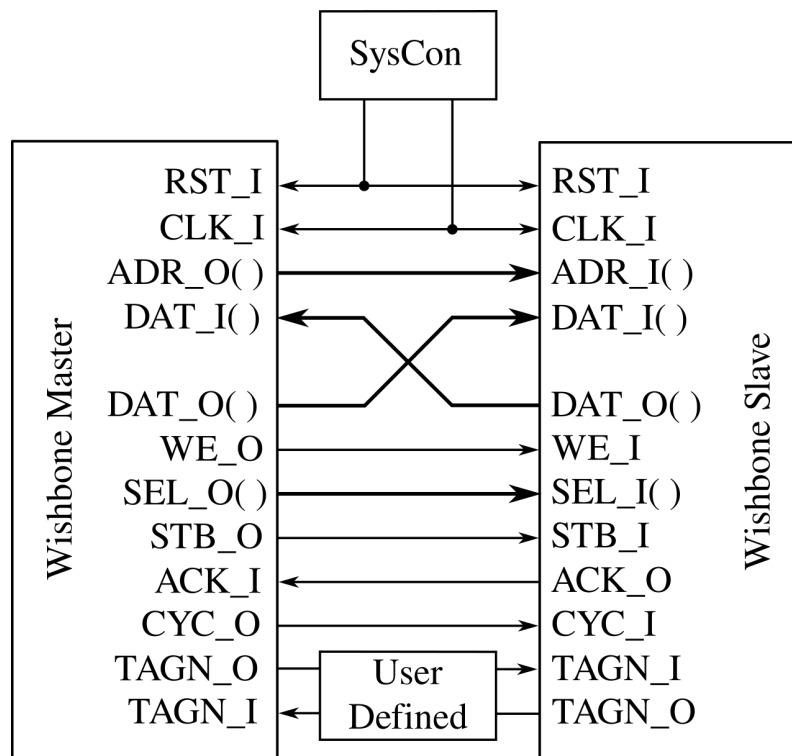


Abbildung 2: **Diagramm des Wishbone-Systembus Schnittstelle.**

Der **Wishbone-Systembus** ist ein offener Busstandard, der die Kommunikation zwischen verschiedenen Hardwaremodulen innerhalb eines Systems ermöglicht. Wie in Abbildung 2 dargestellt, besitzen sowohl der Master als auch die Slaves eigene Anschlüsse für die Datenübertragung. Im PicoNut übernimmt die *Membrana* die Rolle des Masters und steuert den Datenaustausch, während der SDRAM-Controller als Slave auf die Anfragen des Masters reagiert.

Eine Busübertragung beginnt damit, dass der Master die Zieladresse auf **ADR_0** ausgibt und mithilfe der Steuersignale **CYC_0** und **STB_0** einen Buszugriff anfordert. Über das Signal **WE_0** wird festgelegt, ob ein Lese- oder Schreibzugriff durchgeführt werden soll. Bei einem Schreibzugriff werden die zu übertragenden Daten über **DAT_0** an den Slave gesendet. Bei einem Lesezugriff stellt der Slave die angeforderten Daten über **DAT_I** bereit. Sobald der Zugriff abgeschlossen ist, signalisiert der Slave dies durch das Signal **ACK_I**, wodurch die Busübertragung beendet wird.

2.2.3. SDRAM

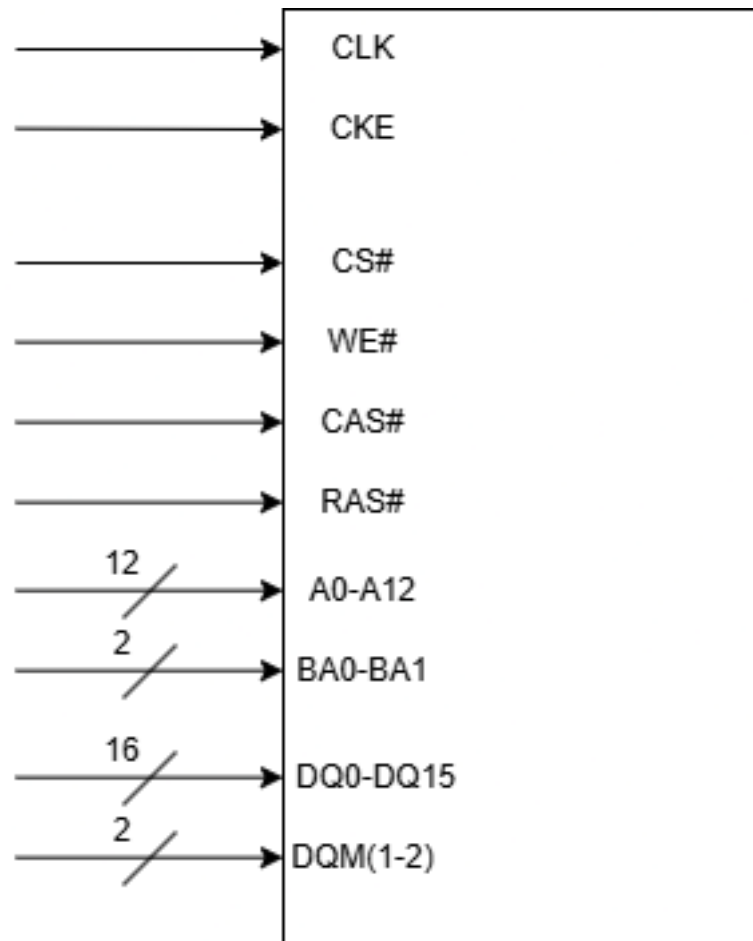


Abbildung 3: **Anschlüsse des SDRAMs.**

Der **SDRAM** ist in mehrere Bänke (Banks) unterteilt. Jede Bank besteht aus einer Vielzahl von Zeilen (Rows) und Spalten (Columns). Bevor auf Daten zugegriffen werden kann, müssen zunächst die gewünschte Bank und die entsprechende Zeile aktiviert werden. Die Auswahl erfolgt über die Adressleitungen BA und A.

Damit ein Befehl vom SDRAM ausgeführt wird, müssen die entsprechenden Steuersignale an den Eingängen CS#, WE#, CAS# und RAS# anliegen, wie in [6] dargestellt. Zusätzlich muss der Takt über CLK anliegen und mit CKE aktiviert werden. Das Zeichen # kennzeichnet dabei ein invertiertes, also **aktiv-low**, Signal.

Nach der Aktivierung einer Zeile kann nach Einhaltung der erforderlichen Wartezeit ein Read- oder Write-Befehl ausgeführt werden. Dabei wird über die Adressleitung A die gewünschte Spaltenadresse übergeben. Bei einem Schreibzugriff werden die zu speichernden Daten gleichzeitig über den Datenbus DQ an den SDRAM übertragen. Bei einem Lesezugriff stellt der SDRAM die angeforderten Daten nach einer festgelegten Verzögerung auf dem Datenbus DQ bereit.

Nach Abschluss der Speicherzugriffe sollte die geöffnete Zeile mit dem Befehl Precharge wieder geschlossen werden. Hierfür müssen erneut die entsprechenden Steuersignale anliegen.

Da ein SDRAM ein dynamischer Speicher ist, werden die Daten in Kondensatoren gespeichert. Diese verlieren mit der Zeit ihre Ladung, wodurch die gespeicherten Informationen verloren gehen würden. Aus diesem Grund müssen die Speicherzellen in regelmäßigen Zeitabständen mit einem Refresh-Befehl aufgefrischt werden, um die gespeicherten Daten zu erhalten.

Der in dieser Arbeit verwendete SDRAM besitzt eine Speicherkapazität von 32 MB. Daraus ergibt sich ein adressierbarer Speicherbereich von 33.554.432 Byte. Die Speicheradresse setzt sich dabei aus der Bank-, Zeilen- und Spaltenadresse zusammen und wird vom SDRAM-Controller entsprechend auf die Adressleitungen BA und A aufgeteilt.

2.3. Entwurf des SDRAM-Controllers

Wie bereits in Abschnitt 2.2 beschrieben, empfängt der SDRAM-Controller als Slave die Steuer- und Datensignale über den Wishbone-Systembus. Der SDRAM kann diese Signale jedoch nicht direkt verarbeiten, da er eine eigene Befehls- und Signalstruktur besitzt. Daher wird ein Modul benötigt, das die Signale des Wishbone-Busses in die entsprechenden SDRAM-Befehle umsetzt und dabei die erforderlichen Wartezeiten sowie das zeitliche Verhalten des Speichers berücksichtigt.

Zu diesem Zweck wurde der SDRAM-Controller entwickelt. Er bildet die Schnittstelle zwischen dem Wishbone-Bus und dem SDRAM und übernimmt die Übersetzung der Buszugriffe in die entsprechenden SDRAM-Kommandos. Die Implementierung dieser Arbeit orientiert sich an den Grundlagen der auf fpga4fun.com veröffentlichten SDRAM-Implementierung [1], erweitert diese jedoch um die für den PicoNut erforderliche Wishbone-Anbindung sowie die vollständige synthesefähige Integration.

2.3.1. Einfache SDRAM-Controller Zustandsmaschine

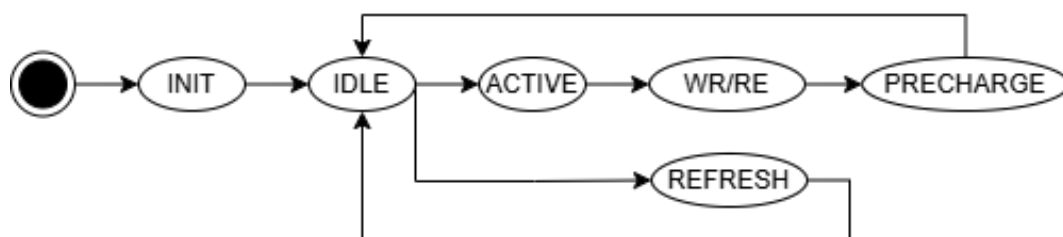


Abbildung 4: **Einfache Zustandsmaschine** [1]

Diese Zustandsmaschine ist vergleichsweise einfach aufgebaut, da lediglich die vom SDRAM vorgegebenen Wartezeiten zwischen den einzelnen Befehlen eingehalten werden müssen. Nach dem Start wird zunächst der Zustand INIT ausgeführt, in dem der SDRAM initialisiert wird. Anschließend wechselt die Zustandsmaschine in den Zustand IDLE. In diesem Zustand wartet sie auf ein Ereignis. Entweder sendet die Membrana über den Wishbone-Systembus einen Lese- oder Schreibzugriff, oder ein interner Zähler löst einen Refresh des SDRAM aus.

Geht eine Anfrage über den *Wishbone-Systembus* ein, wird zunächst überprüft, ob es sich um einen Lese- oder Schreibzugriff handelt. Trifft eines von beiden zu, wechselt die Zustandsmaschine in den Zustand ACTIVE. In diesem Zustand werden die gewünschte Bank und die entsprechende Zeile des *SDRAM* aktiviert.

Anschließend erfolgt der Wechsel in den Zustand READ oder WRITE. Für eine übersichtlichere Darstellung werden diese beiden Zustände in der Einleitung gemeinsam betrachtet, obwohl sie in der Implementierung getrennt realisiert sind. Da der *PicoNut* ein *32-Bit-Prozessor* ist und der verwendete *SDRAM* über einen *16-Bit-Datenbus* verfügt, muss jeder Lese- und Schreibzugriff in zwei aufeinanderfolgenden Übertragungen erfolgen.

Nachdem alle Daten übertragen wurden, wechselt die Zustandsmaschine in den Zustand PRECHARGE. In diesem Zustand wird die geöffnete Bank wieder geschlossen und die gelesenen Daten werden an die *Membrana* zurückgegeben. Anschließend erfolgt der Wechsel zurück in den Zustand IDLE.

Läuft währenddessen der interne Refresh-Zähler ab, wird aus dem Zustand IDLE in den Zustand REFRESH gewechselt. Dort wird der entsprechende Refresh-Befehl an den *SDRAM* gesendet, damit die gespeicherten Daten erhalten bleiben. Nach Abschluss des Refresh-Vorgangs kehrt die Zustandsmaschine wieder in den Zustand IDLE zurück und wartet auf das nächste Ereignis.

Zwischen den einzelnen Befehlen befinden sich zusätzlich NOP-Zustände. Diese dienen dazu, die vom *SDRAM* geforderten Wartezeiten einzuhalten, sodass die Befehle zuverlässig ausgeführt werden können. Je nach Befehl sind ein oder zwei NOP-Zustände erforderlich. Zur besseren Übersicht werden diese Zustände im weiteren Verlauf der Arbeit nicht gesondert betrachtet.

2.3.2. Erforderliche Befehle und Signal Zustände

NAME (FUNCTION)	CS#	RAS#	CAS#	WE#	DQM#	ADDR	DQ
NO OPERATION (NOP)	L	H	H	H	X	X	X
ACTIVE	L	L	H	H	X	Bank/Row	X
READ	L	H	L	H	L/H _s	Bank/Col	X
WRITE	L	H	L	L	L/H _s	Bank/Col	Valid
BURST TERMINATE	L	H	H	L	X	X	Active
PRECHARGE	L	L	H	L	X	Code	X
AUTO REFRESH	L	L	L	H	X	X	X
LOAD MODE REGISTER	L	L	L	L	X	Op-Code	X

Tabelle 1: **Commandos für den SDRAM [6]**

Die Tabelle 1 zeigt die Signalbelegung der wichtigsten SDRAM-Befehle. Für jeden Befehl ist dargestellt, welche Pegel an den Steuersignalen anliegen müssen, damit der entsprechende Befehl ausgeführt wird. Wie bereits in Abschnitt 2.2 erläutert, kennzeichnet das

Symbol # ein **aktiv-low**-Signal. Das bedeutet, dass ein Signal bei einem Low-Pegel aktiv und bei einem High-Pegel inaktiv ist.

Um Wiederholungen zu vermeiden, werden die einzelnen Eingänge und deren Signalbelegung in den folgenden Beschreibungen der Zustandsmaschine nicht erneut erläutert. Stattdessen wird an den entsprechenden Stellen auf Tabelle 1 verwiesen.

In der ersten Zeile der Tabelle sind die einzelnen Eingänge des SDRAM aufgeführt:

- CS# – Chip Select
- RAS# – Row Address Strobe
- CAS# – Column Address Strobe
- WE# – Write Enable
- DQM – Data Mask
- ADDR – Address
- DQ – Bidirektionaler Dateneingang bzw. Datenausgang

2.3.3. Initialisierung



Abbildung 5: **Zustände im Init Zyklus**

Der Zustand INIT aus Abbildung 4 dient zur Initialisierung des SDRAM. Nach dem Einschalten muss der SDRAM zunächst für mindestens 100 µs mit Spannung versorgt und getaktet werden, bevor Befehle an ihn gesendet werden dürfen. Diese Wartezeit wird im Zustand INIT aus Abbildung 5 eingehalten.

Nach Ablauf der Wartezeit wechselt die Zustandsmaschine in den Zustand PRECHARGE. Dadurch werden alle geöffneten Bänke geschlossen. Dieser Schritt ist notwendig, da sich die Speicherzellen nach dem Einschalten in einem undefinierten Zustand befinden können.

Anschließend werden zwei REFRESH-Zyklen ausgeführt. Diese dienen dazu, den SDRAM in einen definierten Betriebszustand zu versetzen und alle Speicherzellen auf ihre erforderliche Ladung zu bringen.

Im letzten Schritt der Initialisierung wird das LOAD MODE REGISTER-Kommando ausgeführt. Dabei wird das Mode-Register des SDRAM konfiguriert. Für diese Arbeit wurde der Wert 0000000110001 gewählt, da er eine einfache und zuverlässige Ansteuerung des SDRAM ermöglicht.

Die einzelnen Bits des Mode-Registers besitzen dabei folgende Bedeutung [6]:

- A2–A0 : 001 – Burst Length = 2 Wörter
- A3 : 0 – Burst Type = Sequential
- A6–A4 : 011 – CAS Latency = 3
- A8–A7 : 00 – Operating Mode = Standard Operation
- A9 : 0 – Programmed Burst Read / Burst Write
- A12–A10 : 000 – Reserviert

2.3.4. Schreibe- und Lese Zyklen

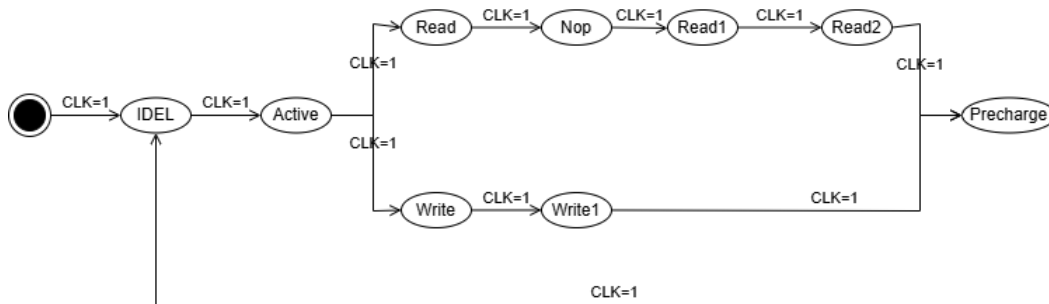


Abbildung 6: Zustände im Lese- und Schreib Zyklen

Im Abbildung 5 wurde der Initialisierungsvorgang beschrieben. Im Folgenden werden der Schreib- und der Lesevorgang erläutert.

Im Zustand IDLE wartet die Zustandsmaschine auf Signale des *Wishbone-Systembus*. Sobald eine Anfrage eingeht, wird ausgewertet, ob es sich um einen Lese- oder Schreibzugriff handelt.

Handelt es sich um einen Schreibzugriff, wechselt die Zustandsmaschine in den Zustand ACTIVE. In diesem Zustand werden die in Tabelle 1 beschriebenen Steuersignale an den SDRAM angelegt. Gleichzeitig werden über die Adressleitungen die Bank- und Zeilenadresse übertragen, wodurch die entsprechende Speicherzeile für den Zugriff aktiviert wird.

Anschließend erfolgt der Wechsel in den Zustand WRITE. Hier wird die Spaltenadresse ausgewählt und gleichzeitig das erste 16-Bit-Datenwort über den Datenbus DQ an den SDRAM übertragen. Danach wechselt die Zustandsmaschine in den Zustand WRITE 1. Dieser stellt einen NOP-Zustand dar. Da der SDRAM im gewählten Modus eine Burst-Länge von zwei Wörtern besitzt, wird während dieses Zustands automatisch das zweite 16-Bit-Datenwort in die nächste Speicheradresse geschrieben. Dadurch sind keine zusätzlichen Zustände wie ACTIVE oder WRITE erforderlich.

Nach Abschluss des Schreibzugriffs wird in den Zustand PRECHARGE gewechselt. In diesem Zustand wird die zuvor geöffnete Zeile wieder geschlossen. Gleichzeitig sendet der SDRAM-Controller über den *Wishbone-Systembus* das ACK-Signal, um den erfolgreichen Abschluss des Schreibzugriffs zu bestätigen.

Erfolgt im Zustand IDLE ein Lesezugriff, wird zunächst ebenfalls in den Zustand ACTIVE gewechselt und die gewünschte Bank sowie die entsprechende Zeile aktiviert. Anschließend folgt der Zustand READ, in dem der Lesevorgang durch die entsprechenden Steuersignale gestartet wird.

Im Gegensatz zu einem Schreibzugriff stehen die Daten jedoch nicht sofort zur Verfügung. Aufgrund der eingestellten CAS-Latenz vergehen zunächst mehrere Taktzyklen, bis der SDRAM die Daten auf dem Datenbus DQ bereitstellt. Diese Wartezeit wird durch die NOP-Zustände im Abbildung 6 realisiert.

Sobald die Wartezeit abgelaufen ist, werden im Zustand READ 1 die ersten 16 Bit des Datenwortes übernommen. Im darauffolgenden Zustand READ 2 werden die restlichen 16

Bit gelesen. Anschließend werden beide Datenhälften zu einem 32-Bit-Wert zusammengesetzt.

Zum Abschluss wechselt die Zustandsmaschine erneut in den Zustand PRECHARGE. Wie bereits beim Schreibzugriff wird die geöffnete Zeile geschlossen. Zusätzlich werden die gelesenen 32 Bit zusammen mit dem ACK-Signal über den *Wishbone-Systembus* an die Membrana zurückgegeben.

2.3.5. Refresh Zyklus

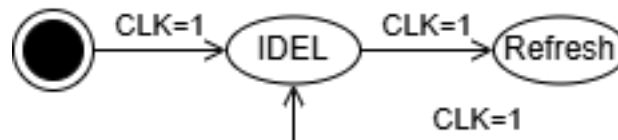


Abbildung 7: **Zustände im Refresh Zyklus**

Im Gegensatz zu Abbildung 6 wird in diesem Fall nicht auf einen Zugriff über den *Wishbone-Systembus* gewartet, sondern auf einen internen Zähler. Dieser löst regelmäßig einen Refresh-Zyklus aus, damit alle Speicherzellen innerhalb von 66 ms aufgefrischt werden. Daraus ergibt sich ein Refresh-Intervall von 7,8 μ s. Bei einem Systemtakt von 25 MHz entspricht dies einem Abstand von 195 Taktzyklen zwischen zwei Refresh-Befehlen.

Für einen gültigen Refresh-Zyklus muss zuvor die geöffnete Zeile mit dem Befehl PRECHARGE geschlossen worden sein. Zwischen PRECHARGE und REFRESH müssen dabei mindestens zwei Taktzyklen liegen. Ebenso muss der REFRESH-Befehl selbst mindestens zwei Taktzyklen aktiv bleiben, damit der SDRAM den Refresh ordnungsgemäß ausführen kann.

Zusätzlich ist zu berücksichtigen, dass sich der *SDRAM-Controller* zum Zeitpunkt des Refreshs bereits in einem Lese- oder Schreibzugriff befinden kann. In diesem Fall muss zunächst der aktuelle Speicherzugriff vollständig beendet werden, bevor der Refresh gestartet werden kann. Aus diesem Grund wird der interne Refresh-Zähler etwas früher ausgelöst als die theoretisch erforderlichen 195 Taktzyklen, sodass der Refresh trotz möglicher Verzögerungen innerhalb des zulässigen Zeitfensters erfolgt.

2.4. Implementierungsdetails des SDRAM-Controllers

Die Implementierung erfolgt entsprechend der in Abschnitt 2.3 beschriebenen Befehlsabfolge. Daher werden in diesem Kapitel ausschließlich die bisher nicht erläuterten Module sowie deren Zusammenspiel mit dem *Wishbone-Systembus* beschrieben.

Wie in Listing 1 dargestellt, sind mehr Zustände vorhanden als in Abschnitt 2.3 beschrieben wurden. Dabei handelt es sich um zusätzliche NOP-Zustände. Alle Zustände, deren Name *before* oder *after* enthält, stellen solche NOP-Zustände dar. Diese Bezeichnung wurde gewählt, da sie den jeweiligen Zweck des Zustands besser beschreibt und die Zustandsmaschine übersichtlicher macht, als eine fortlaufende Benennung wie NOP 1 bis NOP 10.

```

1  enum e_wb_state {
2      SD_INIT_WAIT = 0,          //0
3      SD_INIT_PRECHARGE,        //1
4      SD_INIT_REFRESH,          //2
5      SD_INIT_REFRESH_2,        //3
6      SD_INIT_NOP_1,            //4
7      SD_INIT_NOP_2,            //5
8      SD_INIT_NOP_3,            //6
9      SD_INIT_NOP_4,            //7
10     SD_INIT_MODE,              //8
11     SD_INIT_IDEL,              //9
12     WB_IDLE,                   //10
13     WB_READ,                   //11
14     SD_ACTIVE_FOR_READ_0,      //12
15     SD_ACTIVE_FOR_WRITE_0,     //13
16     SD_ACTIVE_FOR_READ_1,      //14
17     SD_ACTIVE_FOR_WRITE_1,     //15
18     SD_NOP_AFTER_AKTIVE_READ_1, //16
19     SD_NOP_AFTER_AKTIVE_READ_2, //17
20     SD_NOP_AFTER_AKTIVE_READ_3, //18
21     SD_NOP_AFTER_AKTIVE_WRITE_1, //19
22     SD_NOP_AFTER_AKTIVE_WRITE_2, //20
23     SD_NOP_AFTER_AKTIVE_WRITE_3, //21
24     SD_READ,                   //22
25     SD_WRITE,                  //23
26     SD_NOP_AFTER_READ_1,       //24
27     SD_NOP_AFTER_READ_2,       //25
28     SD_NOP_AFTER_READ_3,       //26
29     SD_NOP_AFTER_READ_4,       //27
30     SD_NOP_AFTER_WRITE_1,       //28
31     SD_NOP_AFTER_WRITE_2,       //29
32     SD_PRECHARGE,              //30
33     SD_NOP_AFTER_PRECHARGE_1,  //31
34     SD_NOP_AFTER_PRECHARGE_2,  //32
35     SD_NOP_FOR_REFRESH,        //33
36     SD_REFRESH,                //34
37     SD_NOP_AFTER_REFRESH       //35
38 };

```

Listing 1: Zustände im Code

2.4.1. Idel Zustand

Ein genauerer Blick auf den Zustand IDLE zeigt den Ablauf der Anfrageverarbeitung. Zunächst wird überprüft, ob der interne Refresh-Zähler abgelaufen ist. Ist dies der Fall, wird unmittelbar in den REFRESH-Zustand gewechselt.

Ist kein Refresh erforderlich, wertet der *SDRAM-Controller* die Signale STB_I und CYC_I des *Wishbone-Systembus* aus. Sind beide Signale aktiv, wird anschließend überprüft, ob die am Eingang ADR_I anliegende Adresse in den für den *SDRAM-Controller* vorgesehenen Adressbereich fällt. Trifft dies zu, wird anhand des Signals WE_I bestimmt, ob es sich um einen Lese- oder Schreibzugriff handelt. Abhängig vom Ergebnis erfolgt anschließend der Wechsel in den entsprechenden ACTIVE-Zustand, von dem aus der jeweilige Speicherzugriff eingeleitet wird.

```
1  if (c_rfrsh_strt.read() == 1)
2      {
3          c_next_state = SD_NOP_FOR_REFRESH;
4
5      }else if(wb_slave.stb_i.read() == 1 && wb_slave.cyc_i.read()
6              == 1)
7      {
8          if((PN_CFG_SDRAM_CONTROLLER_BASE_ADDRESS <=
9              wb_slave.adr_i.read()) &&
10             (wb_slave.adr_i.read() <
11              (PN_CFG_SDRAM_CONTROLLER_BASE_ADDRESS +
12              PN_CFG_SDRAM_CONTROLLER_SIZE )))
13          {
14              if(wb_slave.we_i.read() == 1)
15              {
16                  c_next_state = SD_ACTIVE_FOR_WRITE_0;
17              }
18              else
19              {
20                  c_next_state = SD_ACTIVE_FOR_READ_0;
21              }
22          }
23      }
24  }
```

Listing 2: Idel Zustand

2.4.2. Zwischenspeicher

Da bei jedem Speicherzugriff sowohl eine Adresse als auch – im Falle eines Schreibzugriffs – die zu speichernden Daten übernommen werden müssen, werden diese zunächst zwischengespeichert und in die für den SDRAM benötigte Form aufgeteilt. Diese Verarbeitung erfolgt nicht innerhalb der Zustandsmaschine, sondern im Prozess `proc_buffer_write`. Dort werden die eingehenden Daten und Adressen aufbereitet und für die anschließenden Speicherzugriffe bereitgestellt.

```

1  if(c_write_en.read() == 1)
2      {
3          c_row_buf  = wb_slave.adr_i.read().range(24,13);
4          c_ba_buf   = wb_slave.adr_i.read().range(12,11);
5          c_col_buf  = wb_slave.adr_i.read().range(10,2);
6          c_dq_buf_1 = wb_slave.dat_i.read().range(15,0);
7          c_dq_buf_2 = wb_slave.dat_i.read().range(31,16);
8
9      }else{
10         if(c_rd_buf_1.read() == 1)
11             {
12                 c_dq_buf_1 = sdram_dq_i.read();
13             }
14         if(c_rd_buf_2.read() == 1)
15             {
16                 c_dq_buf_2 = sdram_dq_i.read();
17             }
18     }
19
20     if(c_read_en.read() == 1)
21     {
22         c_row_buf  = wb_slave.adr_i.read().range(24,13);
23         c_ba_buf   = wb_slave.adr_i.read().range(12,11);
24         c_col_buf  = wb_slave.adr_i.read().range(10,2);
25     }

```

Listing 3: Daten und Adress Zwischenspeicher

Hier ist gut zu erkennen, wie die eingehenden Adressen und Daten in Zwischenspeichern abgelegt und in das für den *SDRAM* benötigte Format aufgeteilt werden.

Auffällig ist, dass die Adressauswertung erst bei Bit 2 beginnt und nicht bei Bit 0. Der Grund dafür ist, dass der PicoNut ein 32-Bit-Prozessor ist. Ein Speicherzugriff erfolgt daher immer auf ein 32-Bit-Datenwort, also 4 Byte. Dadurch erhöht sich die Adresse bei jedem Zugriff um vier Byte (z. B. 0x0000, 0x0004, 0x0008, ...). Die beiden niederwertigsten Adressbits (A0 und A1) werden somit nicht benötigt und können bei der Adressierung des *SDRAMs* entfallen.

Anschließend wird die verbleibende Adresse in die Bereiche Bank, Row und Column aufgeteilt. Diese Aufteilung ergibt sich aus der Speicherorganisation des verwendeten *SDRAM*. Der Speicher besitzt vier Bänke sowie eine feste Anzahl von Zeilen und Spalten, wodurch sich die vollständige Speicheradresse zusammensetzt.

Da der PicoNut mit 32 Bit arbeitet, der verwendete *SDRAM* jedoch nur einen 16-Bit-Datenbus besitzt, müssen die Daten ebenfalls aufgeteilt werden. Hierzu wird das 32-Bit-

Datenwort in zwei 16-Bit-Wörter zerlegt. Zunächst werden die niederwertigen 16 Bit übertragen, anschließend die höherwertigen 16 Bit. Beim Lesevorgang erfolgt derselbe Ablauf. Die beiden 16-Bit-Werte werden anschließend im Zustand PRECHARGE wieder zu einem 32-Bit-Datenwort zusammengefügt.

2.4.3. Precharge Zustand

```

1  case SD_PRECHARGE:
2
3      wb_slave.dat_o      = sc_uint<32>((c_dq_buf_2.read() << 16 )
      | c_dq_buf_1.read());
4      sdram_cp_sel_n      = 0;
5      sdram_row_ac_sel_n  = 0;
6      sdram_wr_en_n       = 0;
7      wb_slave.ack_o      = 1;
8      sdram_addr          = SDRAM_CONTROLLER_PRECHARGE;
9
10     if(wb_slave.stb_i.read() == 0 && wb_slave.cyc_i.read() == 0)
11     {
12         c_next_state = SD_NOP_AFTER_PRECHARGE_1;
13     }
14     break;

```

Listing 4: Zustand Precharge

Der Zustand PRECHARGE ist besonders erwähnenswert, da er sich in einigen Punkten von der in Abschnitt 2.3 beschriebenen Funktionsweise unterscheidet.

Zum einen werden in diesem Zustand die zuvor gelesenen 16-Bit-Datenwörter zu einem 32-Bit-Datenwort zusammengefügt und direkt auf den Datenausgang des *Wishbone-Systembus* gelegt. Zum anderen wird auf der Adressleitung A10 ein fester Wert von 1 ausgegeben. Dadurch führt der SDRAM einen Precharge-All-Befehl aus, bei dem alle geöffneten Bänke geschlossen werden. Dies stellt sicher, dass sich der Speicher nach jedem Zugriff in einem definierten Zustand befindet.

Ein weiterer Unterschied besteht darin, dass der Übergang in den nächsten Zustand nicht ausschließlich vom Systemtakt abhängt. Stattdessen wartet der SDRAM-Controller, bis die Signale STB_I und CYC_I des *Wishbone-Systembus* wieder inaktiv sind. Erst wenn beide Signale auf 0 zurückgekehrt sind, ist gewährleistet, dass die Membrana die übertragenen Daten beziehungsweise das ACK-Signal übernommen hat. Anschließend wechselt die Zustandsmaschine in den darauffolgenden NOP-Zustand.

2.5. Test und Verifikation des SDRAM-Controllers

Das Testen des SDRAM-Controller stellte sich als eine der größten Herausforderungen dieser Arbeit heraus. Da ein SDRAM feste Timingvorgaben besitzt, musste zunächst ein geeignetes Testverfahren entwickelt werden.

Zu Beginn wurde eine einfache Testbench in *SystemC* erstellt. Dabei zeigte sich jedoch schnell, dass sich nicht alle relevanten Anwendungsfälle und Timingbedingungen

zuverlässig überprüfen ließen. Aus diesem Grund wurde nach einer geeigneten SDRAM-Simulation gesucht. Dabei wurde auf ein Open-Source-Projekt aufmerksam, das ein Verilog-Modell eines SDRAMs bereitstellt. Dieses Modell stammt ursprünglich vom Hersteller *Micron*, ist jedoch auf der offiziellen Webseite nicht mehr verfügbar. Daher wurde die Simulation aus einem Open-Source-Projekt übernommen. Für die verwendeten Lizenzen kann keine Gewähr übernommen werden. Aus diesem Grund wird das Modell nach Abschluss des Projekts wieder aus dem Repository entfernt.

Die größte Schwierigkeit bestand darin, dass das SDRAM-Modell in Verilog vorlag, während der *SDRAM-Controller* in *SystemC* implementiert wurde. Zunächst wurde versucht, das Verilog-Modell nach *SystemC* zu übersetzen. Das verwendete Übersetzungstool konnte jedoch die zeitabhängigen Verhaltensmodelle des SDRAMs nicht korrekt übernehmen. Anschließend wurde versucht, diese Timingbedingungen manuell in *SystemC* nachzubilden, was sich jedoch als zu aufwendig erwies.

Da der PicoNut vor der Synthese ohnehin vollständig nach Verilog übersetzt wird, fiel schließlich die Entscheidung, die Simulation ebenfalls in Verilog durchzuführen. Hierfür musste das bestehende *Makefile*-System erweitert werden, sodass zusätzliche Verilog-Module eingebunden werden konnten. Dies machte es außerdem erforderlich, sich in die Sprache Verilog einzuarbeiten.

Für die Simulation musste der *SDRAM-Controller* geringfügig angepasst werden. Ursprünglich war das Signal *dq* als bidirektionaler Ein-/Ausgang implementiert. Das Übersetzungstool konnte diesen jedoch nicht korrekt verarbeiten. Deshalb wurde das Signal in *dq_i* für den Dateneingang und *dq_o* für den Datenausgang aufgeteilt.

Zusätzlich wurden zwei weitere Verilog-Module implementiert. Das erste Modul ist ein *Tri-State*-Treiber, der die Signale *dq_i* und *dq_o* wieder zu einem bidirektionalen *DQ*-Bus zusammenführt. Das zweite Modul bildet ein Top-Level-Modul, das die korrekte Verschaltung aller Ein- und Ausgänge zwischen dem PicoNut, dem SDRAM und der Simulation übernimmt.

Nachdem alle Module erfolgreich integriert worden waren, wurde eine Testbench in Verilog erstellt. Diese simuliert typische Lese- und Schreibzugriffe über den *Wishbone-Systembus* auf den *SDRAM-Controller*. Die dabei erzeugten Signalverläufe wurden als Waveform-Datei gespeichert und anschließend mit dem Programm *GTKWave* ausgewertet.

Nach erfolgreichem Abschluss der Simulation folgte die Synthese auf der Hardware. Es zeigte sich jedoch, dass eine fehlerfreie Simulation nicht automatisch zu einer funktionierenden Hardwareimplementierung führt. Insbesondere mussten die Timingparameter erneut angepasst werden. Dabei stellte sich heraus, dass eine zu hohe Anzahl an NOP-Zuständen dazu führte, dass der SDRAM nicht mehr korrekt arbeitete. Erst nach mehreren Anpassungen der Wartezeiten konnte ein stabiler Betrieb auf der Hardware erreicht werden.

3. Fazit

3.1. Zusammenfassung

Im Rahmen dieser Arbeit wurde für den PicoNut eine Anbindung an einen größeren externen Speicher entwickelt. Diese wurde mit dem *SDRAM-Controller* realisiert und ermöglicht dem Prozessor den Zugriff auf bis zu 32 MB Speicher.

Bei dem verwendeten Speicher handelt es sich um einen flüchtigen *SDRAM*. Die gespeicherten Daten bleiben daher nur erhalten, solange das Board mit Spannung versorgt wird. Der entwickelte *SDRAM-Controller* unterstützt die Initialisierung des SDRAMs sowie Lese-, Schreib- und Refresh-Zugriffe. Dadurch können Daten während der gesamten Laufzeit des Prozessors zuverlässig gespeichert und wieder ausgelesen werden.

3.2. Ausblick

Da in diesem Modul der Systemtakt verwendet wurde, um die erforderlichen Timings und Zustandswechsel einzuhalten, arbeitet der *SDRAM* mit der gleichen Taktfrequenz wie der PicoNut. Der verwendete *SDRAM* unterstützt jedoch deutlich höhere Taktfrequenzen. Eine mögliche Erweiterung besteht daher darin, einen eigenen Taktgenerator für den *SDRAM* zu implementieren, um die Speicherzugriffe zu beschleunigen.

Darüber hinaus könnte die Zustandsmaschine durch eine komplexere Implementierung weiter optimiert werden. Beispielsweise könnten Speicherzugriffe effizienter geplant oder Wartezeiten reduziert werden. Eine weitere sinnvolle Erweiterung wäre die Unterstützung längerer Burst-Zugriffe, um mehrere aufeinanderfolgende Datenwörter mit nur einem Speicherbefehl übertragen zu können und dadurch den Datendurchsatz zu erhöhen.

Literatur- und Quellenverzeichnis

- [1] „fpga4fun. (o. D.). SDRAM 1 – From static to dynamic“. Zugriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://www.fpga4fun.com/SDRAM1.html>
- [2] „PicoNut - Forschungsgruppe Effiziente Eingebettete Systeme“. Zugriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://ees.tha.de/piconut/index.html>
- [3] „IEEE Standard SystemC(R) Language Reference Manual“, *IEEE Std 1666-2005*, Bd. , S. 1–423, 2006, doi: [10.1109/IEEESTD.2006.99475](https://doi.org/10.1109/IEEESTD.2006.99475).
- [4] „Typst Documentation“. Zugriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://typst.app/docs/>
- [5] „PicoNut Documentation (CC-BY 4.0, <https://creativecommons.org/licenses/by/4.0/>)“. Zugriffen: 4. Juli 2026. [Online]. Verfügbar unter: <https://ees.tha.de/piconut/manual/>
- [6] „Micron Technology (2005): 256Mb: x4, x8, x16 SDRAM – MT48LC16M16A2“. [Online]. Verfügbar unter: <https://www.alldatasheet.com/html-pdf/226490/MICRON/MT48LC16M16A2/222/1/MT48LC16M16A2.html>